Hidden Markov Models and State Space Models: Mathematical Formulation and Algorithms

Lecture Notes

1 Hidden Markov Models: Inference Algorithms

1.1 Computing Marginal Posterior Probabilities

Definition 1.1 (HMM Marginal Posterior)**.** Consider an HMM with state space $\{1, \ldots, K\}$, observation sequence $\mathbf{x}_{1:t} = (x_1, \ldots, x_t)$, and hidden state sequence $\mathbf{s}_{1:t} = (s_1, \ldots, s_t)$. We wish to compute the marginal posterior probability:

$$P(s_t = k | x_1, \ldots, x_t) = \sum_{k_1, \ldots, k_{t-1}} P(s_1 = k_1, \ldots, s_t = k | x_1, \ldots, x_t) \tag{1}$$

Proposition 1.2 (Joint Probability Factorization)**.** *The joint probability in Definition 1.1 can be expressed as:*

$$P(s_1 = k_1, \ldots, s_t = k | x_1, \ldots, x_t) \propto \pi_{k_1} A_{k_1}(x_1) \Phi_{k_1,k_2} A_{k_2}(x_2) \cdots \Phi_{k_{t-1},k} A_k(x_t) \tag{2}$$

where:

- π_k *is the initial state probability for state* k
- $A_k(x)$ *is the emission probability of observation* x *in state* k
- $\Phi_{i,j}$ *is the transition probability from state* i *to state* j

1.1.1 The Naïve Algorithm

Algorithm 1.3 (Naïve Enumeration)**.** The straightforward approach to compute $P(s_t = k | x_{1:t})$ using (1) and (2) is:

1. Start a "bug" at each state $k_1 \in \{1, \ldots, K\}$ at time $t = 1$ holding value $\pi_{k_1} A_{k_1}(x_1)$
2. Move each bug forward in time: make copies of each bug to each subsequent state and multiply the value of each copy by (transition prob.) $\times$ (emission prob.)
3. Repeat until all bugs reach time t
4. Sum up values on all K^{t-1} bugs that reach state $s_t = k$ (one bug per state path)

Remark 1.4 (Exponential Complexity)**.** Algorithm 1.3 has exponential complexity $O(K^T)$ where T is the sequence length, making it computationally infeasible for even moderate sequence lengths.

1.1.2 The Clever Recursion: Dynamic Programming

Algorithm 1.5 (Forward Algorithm with Summation)**.** Instead of maintaining exponentially many bugs as in Algorithm 1.3, at every step replace all bugs at each node with a single bug carrying the sum of values. This is the key insight of dynamic programming for HMMs.

1.2 Forward Recursion and Bayesian Filtering

Definition 1.6 (State Space Model)**.** Consider a state space model with latent states z_t and observations x_t:

$$\begin{array}{ccccccccc} z_1 & \to & z_2 & \to & z_3 & \to \cdots \to & z_T \\ \downarrow & & \downarrow & & \downarrow & & \downarrow \\ x_1 & & x_2 & & x_3 & & x_T \end{array}$$

Theorem 1.7 (Bayesian Filtering Recursion)**.** *For the state space model in Definition 1.6, the posterior distribution over the current state given all past observations satisfies:*

$$\begin{aligned} P(z_t|x_{1:t}) &= \int P(z_t, z_{t-1}|x_t, x_{1:t-1})dz_{t-1} \\ &= \int \frac{P(x_t, z_t, z_{t-1}|x_{1:t-1})}{P(x_t|x_{1:t-1})}dz_{t-1} \\ &\propto \int P(x_t|z_t, z_{t-1}, x_{1:t-1})P(z_t|z_{t-1}, x_{1:t-1})P(z_{t-1}|x_{1:t-1})dz_{t-1} \\ &= \int P(x_t|z_t)P(z_t|z_{t-1})P(z_{t-1}|x_{1:t-1})dz_{t-1} \end{aligned} \tag{3}$$

where the last step uses the Markov property.

Proof. The derivation follows from:

1. Marginalizing over z_{t-1}: $P(z_t|x_{1:t}) = \int P(z_t, z_{t-1}|x_{1:t})dz_{t-1}$
2. Bayes rule: $P(z_t, z_{t-1}|x_{1:t}) = \frac{P(x_t, z_t, z_{t-1}|x_{1:t-1})}{P(x_t|x_{1:t-1})}$
3. Chain rule: $P(x_t, z_t, z_{t-1}|x_{1:t-1}) = P(x_t|z_t, z_{t-1}, x_{1:t-1})P(z_t|z_{t-1}, x_{1:t-1})P(z_{t-1}|x_{1:t-1})$
4. Markov property eliminates conditioning on past: $P(x_t|z_t, z_{t-1}, x_{1:t-1}) = P(x_t|z_t)$ and $P(z_t|z_{t-1}, x_{1:t-1}) = P(z_t|z_{t-1})$

□

Remark 1.8 (Forward Recursion Interpretation)**.** This is a forward recursion based on Bayes rule that allows us to compute the filtering distribution $P(z_t|x_{1:t})$ sequentially as new observations arrive.

1.3 The HMM Forward Pass

Definition 1.9 (Forward Variable)**.** For an HMM with parameters $\theta = \{A, \Phi, \pi\}$, define the forward variable:

$$\alpha_t(i) = P(x_1, \ldots, x_t, s_t = i|\theta) \tag{4}$$

This is the joint probability of the observation sequence up to time t and being in state i at time t.

Theorem 1.10 (Forward Recursion for HMMs)**.** *The forward variables (Definition 1.9) satisfy the recursion:*

$$\begin{aligned} \alpha_1(i) &= \pi_i A_i(x_1) \\ \alpha_{t+1}(i) &= \left(\sum_{j=1}^{K} \alpha_t(j) \Phi_{ji} \right) A_i(x_{t+1}) \end{aligned} \tag{5}$$

Proof. For $t = 1$, we have by Definition 1.9:

$$\alpha_1(i) = P(x_1, s_1 = i|\theta) = P(s_1 = i|\theta) P(x_1|s_1 = i, \theta) = \pi_i A_i(x_1)$$

For the recursive step, we marginalize over the previous state:

$$\begin{aligned} \alpha_{t+1}(i) &= P(x_{1:t+1}, s_{t+1} = i|\theta) \\ &= \sum_{j=1}^{K} P(x_{1:t+1}, s_t = j, s_{t+1} = i|\theta) \\ &= \sum_{j=1}^{K} P(x_{t+1}|s_{t+1} = i, x_{1:t}, s_t = j, \theta) P(s_{t+1} = i|s_t = j, x_{1:t}, \theta) P(x_{1:t}, s_t = j|\theta) \\ &= \sum_{j=1}^{K} A_i(x_{t+1}) \Phi_{ji} \alpha_t(j) \end{aligned}$$

where we used the Markov property in the last step. □

Corollary 1.11 (Posterior from Forward Variables)**.** *From Theorem 1.10, the posterior distribution can be obtained by normalization:*

$$P(s_t = i|x_1, \ldots, x_t, \theta) = \frac{\alpha_t(i)}{\sum_{k=1}^{K} \alpha_t(k)} \tag{6}$$

Corollary 1.12 (Likelihood Computation)**.** *The forward algorithm (Theorem 1.10) enables efficient likelihood computation in $O(TK^2)$ time:*

$$P(x_1, \ldots, x_T|\theta) = \sum_{s_1, \ldots, s_T} P(x_1, \ldots, x_T, s_1, \ldots, s_T|\theta) = \sum_{k=1}^{K} \alpha_T(k) \tag{7}$$

This avoids the exponential number of paths (K^T) in the naïve sum (Algorithm 1.3).

1.4 Kalman Filtering: The Linear Gaussian State Space Model

Definition 1.13 (Linear Gaussian State Space Model)**.** The LGSSM is a special case of Definition 1.6, defined by:

$$\begin{aligned} z_1 &\sim \mathcal{N}(\mu_0, Q_0) \\ z_t|z_{t-1} &\sim \mathcal{N}(Az_{t-1}, Q) \\ x_t|z_t &\sim \mathcal{N}(Cz_t, R) \end{aligned} \tag{8}$$

where A is the state transition matrix, C is the observation matrix, Q is the process noise covariance, and R is the observation noise covariance.

Lemma 1.14 (Gaussian Conditioning)**.** *Let (A, B) be a partition of $\{1, \dots, d\}$ and let $X = (X_A, X_B)$ be a Gaussian random vector in $\mathbb{R}^d = \mathbb{R}^A \times \mathbb{R}^B$ with*

$$\mathbb{E}[X] = \begin{pmatrix} \mu_A \\ \mu_B \end{pmatrix}, \quad Cov[X] = \begin{pmatrix} \Sigma_A & \Sigma_{AB} \\ \Sigma_{AB}^T & \Sigma_B \end{pmatrix}$$

Then the conditional distribution of $X_A|(X_B = x_B)$ is Gaussian with mean

$$\mathbb{E}[X_A|X_B = x_B] = \mu_A + \Sigma_{AB}\Sigma_B^{-1}(x_B - \mu_B) \tag{9}$$

and covariance

$$Cov[X_A|X_B = x_B] = \Sigma_A - \Sigma_{AB}\Sigma_B^{-1}\Sigma_{AB}^T \tag{10}$$

Lemma 1.15 (Woodbury Identity)**.** *For $k \leq n$, invertible matrices $A \in \mathbb{R}^{n\times n}$ and $C \in \mathbb{R}^{k\times k}$, and any $B \in \mathbb{R}^{n\times k}$ and $D \in \mathbb{R}^{k\times n}$:*

$$(A - BCD)^{-1} = A^{-1} + A^{-1}B(C^{-1} + DA^{-1}B)^{-1}DA^{-1} \tag{11}$$

provided A and C are invertible.

Remark 1.16 (Interpretation of Woodbury Identity)**.** In essence: $(A-BCD)^{-1} = f(A^{-1}, C^{-1}, B, D)$. Since C is $k \times k$, the $n \times n$ matrix BCD has rank $\leq k$. Subtracting BCD modifies A on a k-dimensional subspace. Typical use case: We already know A^{-1}, and k is small. We can then invert the $n \times n$ matrix $(A - BCD)$ by inverting only a $k \times k$ matrix.

Theorem 1.17 (Kalman Filter)**.** *For the LGSSM (Definition 1.13), let $\hat{z}_t^\tau = \mathbb{E}[z_t|x_1, \dots, x_\tau]$ and $\hat{V}_t^\tau = Var[z_t|x_1, \dots, x_\tau]$ denote the conditional mean and covariance. Initialize with $\hat{z}_1^0 = \mu_0$ and $\hat{V}_1^0 = Q_0$. Then:*

Prediction step:

$$\begin{aligned} \hat{z}_t^{t-1} &= A\hat{z}_{t-1}^{t-1} \\ \hat{V}_t^{t-1} &= A\hat{V}_{t-1}^{t-1}A^T + Q \end{aligned} \tag{12}$$

Update step:

$$\begin{aligned} K_t &= \hat{V}_t^{t-1}C^T(C\hat{V}_t^{t-1}C^T + R)^{-1} \quad \textit{(Kalman gain)} \\ \hat{z}_t^t &= \hat{z}_t^{t-1} + K_t(x_t - C\hat{z}_t^{t-1}) \\ \hat{V}_t^t &= \hat{V}_t^{t-1} - K_tC\hat{V}_t^{t-1} \end{aligned} \tag{13}$$

Proof. For $t = 1$, we need to compute $P(z_1|x_1)$. By Bayes rule:

$$P(z_1|x_1) \propto P(x_1|z_1)P(z_1) = \mathcal{N}(x_1; Cz_1, R)\mathcal{N}(z_1; \mu_0, Q_0)$$

This is a product of Gaussians, which is Gaussian. We can write the joint distribution:

$$\begin{pmatrix} z_1 \\ x_1 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_0 \\ C\mu_0 \end{pmatrix}, \begin{pmatrix} Q_0 & Q_0C^T \\ CQ_0 & CQ_0C^T + R \end{pmatrix}\right)$$

Applying Lemma 1.14:

$$\begin{aligned} \hat{z}_1^1 &= \mu_0 + Q_0C^T(CQ_0C^T + R)^{-1}(x_1 - C\mu_0) = \hat{z}_1^0 + K_1(x_1 - C\hat{z}_1^0) \\ \hat{V}_1^1 &= Q_0 - Q_0C^T(CQ_0C^T + R)^{-1}CQ_0 = \hat{V}_1^0 - K_1C\hat{V}_1^0 \end{aligned}$$

where $K_1 = Q_0C^T(CQ_0C^T + R)^{-1} = \hat{V}_1^0C^T(C\hat{V}_1^0C^T + R)^{-1}$.

For the recursion, the prediction step follows from the Bayesian filtering recursion (Theorem 1.7):

$$P(z_t|x_{1:t-1}) = \int P(z_t|z_{t-1})P(z_{t-1}|x_{1:t-1})dz_{t-1}$$

Since $z_t|z_{t-1} \sim \mathcal{N}(Az_{t-1}, Q)$ and $z_{t-1}|x_{1:t-1} \sim \mathcal{N}(\hat{z}_{t-1}^{t-1}, \hat{V}_{t-1}^{t-1})$:

$$z_t|x_{1:t-1} \sim \mathcal{N}(A\hat{z}_{t-1}^{t-1}, A\hat{V}_{t-1}^{t-1}A^T + Q)$$

The update step follows the same argument as for $t = 1$ (by Lemma 1.14), replacing the prior with the prediction distribution. □

Remark 1.18 (Kalman Gain Interpretation)**.** The Kalman gain K_t can be interpreted as:

$$K_t = \frac{\langle zx^T \rangle}{\langle xx^T \rangle} = \hat{V}_t^{t-1}C^T(C\hat{V}_t^{t-1}C^T + R)^{-1}$$

where the angle brackets denote expectations under the predictive distribution.

2 Smoothing Algorithms

2.1 Bayesian Smoothing

Definition 2.1 (Smoothing Posterior)**.** The smoothing problem is to compute the marginal posterior:

$$P(z_t|x_{1:T}) \quad \text{for } t < T \tag{14}$$

that is, the distribution over a past state given all observations (past and future).

Theorem 2.2 (Smoothing Decomposition)**.** *The marginal smoothing distribution (Definition 2.1) can be decomposed as:*

$$\begin{aligned} P(z_t|x_{1:T}) &= \frac{P(z_t, x_{t+1:T}|x_{1:t})}{P(x_{t+1:T}|x_{1:t})} \\ &= \frac{P(x_{t+1:T}|z_t)P(z_t|x_{1:t})}{P(x_{t+1:T}|x_{1:t})} \end{aligned} \tag{15}$$

Proof. By Bayes rule:

$$P(z_t|x_{1:T}) = \frac{P(z_t, x_{1:T})}{P(x_{1:T})} = \frac{P(z_t, x_{1:t}, x_{t+1:T})}{P(x_{1:T})}$$

We can rewrite the numerator:

$$\begin{aligned} P(z_t, x_{1:t}, x_{t+1:T}) &= P(x_{t+1:T}|z_t, x_{1:t})P(z_t, x_{1:t}) \\ &= P(x_{t+1:T}|z_t)P(z_t|x_{1:t})P(x_{1:t}) \end{aligned}$$

where we used the Markov property (Definition 1.6): $P(x_{t+1:T}|z_t, x_{1:t}) = P(x_{t+1:T}|z_t)$.

The denominator is:

$$P(x_{1:T}) = P(x_{t+1:T}|x_{1:t})P(x_{1:t})$$

Combining gives the result. □

Remark 2.3 (Forward-Backward Messages)**.** The marginal posterior in Theorem 2.2 combines:

- A **forward message** $P(z_t|x_{1:t})$ (from the filtering algorithm, Theorem 1.7)
- A **backward message** $P(x_{t+1:T}|z_t)$ (propagated backwards from future observations)

2.2 The HMM Forward-Backward Algorithm

Definition 2.4 (Forward and Backward Variables)**.** For an HMM, define:

$$\begin{aligned} \alpha_t(i) &= P(x_{1:t}, s_t = i|\theta) \quad \text{(forward variable, cf. Definition 1.9)} \\ \beta_t(i) &= P(x_{t+1:T}|s_t = i, \theta) \quad \text{(backward variable)} \end{aligned} \tag{16}$$

Theorem 2.5 (Backward Recursion)**.** *The backward variables (Definition 2.4) satisfy:*

$$\begin{aligned} \beta_T(i) &= 1 \\ \beta_t(i) &= \sum_{j=1}^{K} \Phi_{ij} A_j(x_{t+1}) \beta_{t+1}(j) \end{aligned} \tag{17}$$

Proof. The initialization $\beta_T(i) = 1$ follows since there are no future observations after time T.

For the recursion:

$$\begin{aligned} \beta_t(i) &= P(x_{t+1:T}|s_t = i) \\ &= \sum_{j=1}^{K} P(s_{t+1} = j, x_{t+1}, x_{t+2:T}|s_t = i) \\ &= \sum_{j=1}^{K} P(s_{t+1} = j|s_t = i) P(x_{t+1}|s_{t+1} = j) P(x_{t+2:T}|s_{t+1} = j) \\ &= \sum_{j=1}^{K} \Phi_{ij} A_j(x_{t+1}) \beta_{t+1}(j) \end{aligned}$$

where we used the Markov property and conditional independence. □

Theorem 2.6 (Marginal Posterior via Forward-Backward)**.** *Using the forward and backward variables (Definition 2.4), the marginal posterior is given by:*

$$\gamma_t(i) := P(s_t = i|x_{1:T}) = \frac{P(s_t = i, x_{1:t}) P(x_{t+1:T}|s_t = i)}{P(x_{1:T})} = \frac{\alpha_t(i)\beta_t(i)}{\sum_{j=1}^{K} \alpha_t(j)\beta_t(j)} \tag{18}$$

Proof. By the smoothing decomposition (Theorem 2.2):

$$P(s_t = i|x_{1:T}) = \frac{P(x_{t+1:T}|s_t = i) P(s_t = i|x_{1:t}) P(x_{1:t})}{P(x_{1:T})}$$

Now:

$$\begin{aligned} P(s_t = i|x_{1:t}) P(x_{1:t}) &= P(s_t = i, x_{1:t}) = \alpha_t(i) \\ P(x_{t+1:T}|s_t = i) &= \beta_t(i) \\ P(x_{1:T}) &= \sum_{j=1}^{K} P(s_t = j, x_{1:t}) P(x_{t+1:T}|s_t = j) = \sum_{j=1}^{K} \alpha_t(j)\beta_t(j) \end{aligned}$$

□

Remark 2.7 (Probability Flow Interpretation)**.** $\alpha_t(i)$ gives the total inflow of probabilities to node (t, i); $\beta_t(i)$ gives the total outflow of probabilities. The product $\alpha_t(i)\beta_t(i)$ gives the total probability flow through state i at time t.

2.3 Viterbi Decoding

Definition 2.8 (MAP State Sequence)**.** The Viterbi algorithm computes the most probable state sequence:

$$\mathbf{s}^* = \arg\max_{s_{1:T}} P(s_{1:T}|x_{1:T}, \theta) \tag{19}$$

Remark 2.9 (Distinction from Forward-Backward)**.** The forward-backward algorithm (Theorem 2.6) computes marginals $\gamma_t(i)$ which give the most probable state at each time independently. Choosing $i_t^* = \arg\max_i \gamma_t(i)$ for each t gives the path with maximum expected number of correct states. However, this may not be the single path with highest probability (it may even have probability zero!).

Theorem 2.10 (Viterbi Algorithm)**.** *Define:*

$$\delta_t(i) = \max_{s_1,\ldots,s_{t-1}} P(s_1, \ldots, s_{t-1}, s_t = i, x_{1:t}|\theta) \tag{20}$$

Then:

$$\begin{aligned} \delta_1(i) &= \pi_i A_i(x_1) \\ \delta_{t+1}(i) &= \left[\max_j \delta_t(j)\Phi_{ji}\right] A_i(x_{t+1}) \end{aligned} \tag{21}$$

The most probable sequence (Definition 2.8) is recovered by backtracking through the maximizing arguments.

Proof. The recursion follows from dynamic programming:

$$\begin{aligned} \delta_{t+1}(i) &= \max_{s_{1:t}} P(s_{1:t}, s_{t+1} = i, x_{1:t+1}|\theta) \\ &= \max_{s_{1:t}} P(x_{t+1}|s_{t+1} = i)P(s_{t+1} = i|s_t)P(s_{1:t}, x_{1:t}|\theta) \\ &= A_i(x_{t+1}) \max_{s_{1:t}} \Phi_{s_t,i} P(s_{1:t}, x_{1:t}|\theta) \\ &= A_i(x_{t+1}) \max_j \left[\Phi_{ji} \max_{s_{1:t-1}} P(s_{1:t-1}, s_t = j, x_{1:t}|\theta)\right] \\ &= A_i(x_{t+1}) \max_j \left[\Phi_{ji}\delta_t(j)\right] \end{aligned}$$

□

Remark 2.11 (Relationship to Forward Algorithm)**.** The Viterbi recursion (21) looks identical to the forward recursion (5), except with max instead of $\sum$. This is Bellman's dynamic programming algorithm applied to the HMM inference problem.

Remark 2.12 (Bug Interpretation)**.** Using the bug metaphor from Algorithm 1.3: the same trick as forward-backward, except at each step kill all bugs but the one with the highest value at each node.

2.4 Kalman Smoother

Theorem 2.13 (Kalman Smoother)**.** *For the LGSSM (Definition 1.13), the Kalman smoother uses a different decomposition of the smoothing posterior (Definition 2.1):*

$$P(z_t|x_{1:T}) = \int P(z_t|z_{t+1}, x_{1:T})P(z_{t+1}|x_{1:T})dz_{t+1} = \int P(z_t|z_{t+1}, x_{1:t})P(z_{t+1}|x_{1:T})dz_{t+1}$$

where the second equality uses the Markov property.

This gives the backward recursion:

$$\begin{aligned} J_t &= \hat{V}_t^t A^T (\hat{V}_{t+1}^t)^{-1} \\ \hat{z}_t^T &= \hat{z}_t^t + J_t(\hat{z}_{t+1}^T - A\hat{z}_t^t) \\ \hat{V}_t^T &= \hat{V}_t^t + J_t(\hat{V}_{t+1}^T - \hat{V}_{t+1}^t)J_t^T \end{aligned} \tag{22}$$

where $\hat{z}_t^t$ and $\hat{V}_t^t$ are the filtered estimates from the Kalman filter (Theorem 1.17).

Proof. The joint distribution of $(z_t, z_{t+1})|x_{1:t}$ is:

$$\begin{pmatrix} z_t \\ z_{t+1} \end{pmatrix} \Big| x_{1:t} \sim \mathcal{N}\left(\begin{pmatrix} \hat{z}_t^t \\ A\hat{z}_t^t \end{pmatrix}, \begin{pmatrix} \hat{V}_t^t & \hat{V}_t^t A^T \\ A\hat{V}_t^t & A\hat{V}_t^t A^T + Q \end{pmatrix} \right)$$

By Lemma 1.14, conditioning on z_{t+1} gives:

$$z_t | z_{t+1}, x_{1:t} \sim \mathcal{N}(\hat{z}_t^t + J_t(z_{t+1} - A\hat{z}_t^t), \hat{V}_t^t - J_t A\hat{V}_t^t)$$

where $J_t = \hat{V}_t^t A^T (\hat{V}_{t+1}^t)^{-1}$.

Taking expectation over $z_{t+1}|x_{1:T}$ (which has mean $\hat{z}_{t+1}^T$ and covariance $\hat{V}_{t+1}^T$) gives:

$$\begin{aligned} \mathbb{E}[z_t|x_{1:T}] &= \hat{z}_t^t + J_t(\hat{z}_{t+1}^T - A\hat{z}_t^t) \\ \mathrm{Var}[z_t|x_{1:T}] &= \hat{V}_t^t - J_t A\hat{V}_t^t + J_t\hat{V}_{t+1}^T J_t^T = \hat{V}_t^t + J_t(\hat{V}_{t+1}^T - \hat{V}_{t+1}^t)J_t^T \end{aligned}$$

□

3 Learning in State Space Models

3.1 Maximum Likelihood Learning via EM

Definition 3.1 (EM for State Space Models)**.** For an SSM (Definition 1.6) with parameters $\theta = \{\mu_0, Q_0, A, Q, C, R\}$, the EM algorithm alternates between:

E-step: Compute $q^*(z_{1:T}) = P(z_{1:T}|x_{1:T}, \theta^{\text{old}})$ using the forward-backward (or Kalman smoothing, Theorem 2.13) algorithm.

M-step: Maximize

$$\mathcal{F}(q, \theta) = \int dz_{1:T} q(z_{1:T}) \left[\log P(x_{1:T}, z_{1:T}|\theta) - \log q(z_{1:T})\right] \tag{23}$$

with respect to θ holding q fixed.

Proposition 3.2 (Joint Likelihood Factorization)**.** *For the LGSSM (Definition 1.13):*

$$P(\mathbf{z}, \mathbf{x}|\theta) = P(z_1)P(x_1|z_1)\prod_{t=2}^{T} P(z_t|z_{t-1})P(x_t|z_t) \tag{24}$$

where all factors are Gaussian, making the M-step tractable as weighted least squares problems.

3.2 M-step for the Observation Matrix

Theorem 3.3 (M-step for C)**.** *In the EM algorithm for the LGSSM (Definition 3.1), the update for the observation matrix C is:*

$$C^{new} = \left(\sum_{t=1}^{T} x_t \langle z_t \rangle^T\right) \left(\sum_{t=1}^{T} \langle z_t z_t^T \rangle\right)^{-1} \tag{25}$$

where expectations are taken under $q^*(z_{1:T})$.

Proof. The observation model (Definition 1.13) is:

$$P(x_t|z_t) \propto \exp\left[-\frac{1}{2}(x_t - Cz_t)^T R^{-1}(x_t - Cz_t)\right]$$

The M-step objective is:

$$C^{\text{new}} = \arg\max_C \left\langle \sum_t \log P(x_t|z_t) \right\rangle_q = \arg\max_C \left\langle -\frac{1}{2}\sum_t (x_t - Cz_t)^T R^{-1}(x_t - Cz_t) \right\rangle_q$$

Expanding:

$$= \arg\max_C \left\{ -\frac{1}{2}\sum_t \left[x_t^T R^{-1} x_t - 2x_t^T R^{-1} C\langle z_t\rangle + \langle z_t^T C^T R^{-1} C z_t\rangle\right] \right\}$$

$$= \arg\max_C \left\{ \operatorname{Tr}\left[C\sum_t \langle z_t\rangle x_t^T R^{-1}\right] - \frac{1}{2}\operatorname{Tr}\left[C^T R^{-1} C \sum_t \langle z_t z_t^T\rangle\right] \right\}$$

Taking the derivative with respect to C using $\frac{\partial \operatorname{Tr}[AB]}{\partial A} = B^T$:

$$\frac{\partial}{\partial C} = R^{-1}\sum_t x_t \langle z_t\rangle^T - R^{-1} C \sum_t \langle z_t z_t^T\rangle$$

Setting to zero and solving:

$$C^{\text{new}} = \left(\sum_t x_t \langle z_t\rangle^T\right)\left(\sum_t \langle z_t z_t^T\rangle\right)^{-1}$$

□

Remark 3.4 (Weighted Linear Regression)**.** The update (25) has the form of a weighted linear regression where we regress x_t onto z_t, with the expectation accounting for uncertainty in the latent states.

3.3 M-step for the Transition Matrix

Theorem 3.5 (M-step for A)**.** *In the EM algorithm for the LGSSM (Definition 3.1), the update for the transition matrix A is:*

$$A^{new} = \left(\sum_{t=1}^{T-1} \langle z_{t+1} z_t^T\rangle\right)\left(\sum_{t=1}^{T-1} \langle z_t z_t^T\rangle\right)^{-1} \tag{26}$$

Proof. The transition model (Definition 1.13) is:

$$P(z_{t+1}|z_t) \propto \exp\left[-\frac{1}{2}(z_{t+1} - Az_t)^T Q^{-1}(z_{t+1} - Az_t)\right]$$

Following the same derivation as for C (Theorem 3.3):

$$A^{\text{new}} = \arg\max_A \left\langle \sum_{t=1}^{T-1} \log P(z_{t+1}|z_t) \right\rangle_q$$

Expanding and taking derivatives:

$$= \arg\max_A \left\{ \operatorname{Tr}\left[A \sum_{t=1}^{T-1} \langle z_t z_{t+1}^T \rangle Q^{-1}\right] - \frac{1}{2}\operatorname{Tr}\left[A^T Q^{-1} A \sum_{t=1}^{T-1} \langle z_t z_t^T \rangle\right] \right\}$$

$$\frac{\partial}{\partial A} = Q^{-1} \sum_{t=1}^{T-1} \langle z_{t+1} z_t^T \rangle - Q^{-1} A \sum_{t=1}^{T-1} \langle z_t z_t^T \rangle$$

Setting to zero:

$$A^{\text{new}} = \left(\sum_{t=1}^{T-1} \langle z_{t+1} z_t^T \rangle\right)\left(\sum_{t=1}^{T-1} \langle z_t z_t^T \rangle\right)^{-1}$$

□

Remark 3.6 (Cross-Covariance Requirement)**.** Note that the update (26) requires the smoothed cross-covariance $\langle z_{t+1} z_t^T \rangle$, which requires a two-state extension of the Kalman smoother (Theorem 2.13) to compute.

3.4 Online Gradient Learning

Definition 3.7 (Online Learning for SSMs)**.** Time series data must often be processed in real-time. We can update parameters online as observations arrive by updating a local version of the likelihood based on the Kalman filter estimates (Theorem 1.17).

Proposition 3.8 (Incremental Log-Likelihood)**.** *The log-likelihood can be decomposed as:*

$$\ell = \sum_{t=1}^{T} \log P(x_t|x_{1:t-1}) = \sum_{t=1}^{T} \ell_t \tag{27}$$

where

$$\ell_t = -\frac{D}{2}\log 2\pi - \frac{1}{2}\log|\Sigma| - \frac{1}{2}(x_t - C\hat{z}_t^{t-1})^T \Sigma^{-1}(x_t - C\hat{z}_t^{t-1}) \tag{28}$$

with $D = \dim(x)$, $\hat{z}_t^{t-1} = A\hat{z}_{t-1}^{t-1}$ (from the prediction step (12)), $\Sigma = C\hat{V}_t^{t-1}C^T + R$, and $\hat{V}_t^{t-1} = A\hat{V}_{t-1}^{t-1}A^T + Q$.

Algorithm 3.9 (Online Gradient Learning)**.** We differentiate ℓ_t (Proposition 3.8) to obtain gradient rules for A, C, Q, R. The size of the gradient step (learning rate) reflects our expectation about nonstationarity.

3.5 Learning HMMs via EM (Baum-Welch)

Definition 3.10 (EM for HMMs)**.** For an HMM with parameters $\theta = \{\pi, \Phi, A\}$:

E-step: Compute $q^*(s_{1:T}) = P(s_{1:T}|x_{1:T}, \theta^{\text{old}})$ using the forward-backward algorithm (Theorem 2.6). We only need the marginals $q(s_t, s_{t+1})$.

M-step: Maximize

$$\mathcal{F}(q, \theta) = \sum_{s_{1:T}} q(s_{1:T}) \left[\log P(x_{1:T}, s_{1:T}|\theta) - \log q(s_{1:T})\right]$$

Definition 3.11 (Two-Slice Marginal)**.** Define the expected number of transitions from state i to j beginning at time t:

$$\xi_t(i \to j) := P(s_t = i, s_{t+1} = j|x_{1:T}) = \frac{\alpha_t(i)\Phi_{ij}A_j(x_{t+1})\beta_{t+1}(j)}{P(x_{1:T})} \tag{29}$$

where α_t and β_t are the forward and backward variables (Definition 2.4).

Theorem 3.12 (Baum-Welch Parameter Updates)**.** *The M-step updates for the EM algorithm (Definition 3.10) are:*

Initial distribution:

$$\hat{\pi}_i = \gamma_1(i) \tag{30}$$

Transition probabilities:

$$\hat{\Phi}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i \to j)}{\sum_{t=1}^{T-1} \gamma_t(i)} \tag{31}$$

where ξ_t is the two-slice marginal (Definition 3.11) and γ_t is the marginal posterior (18)*.*

Emission probabilities (discrete):

$$\hat{A}_{ik} = \frac{\sum_{t:x_t=k} \gamma_t(i)}{\sum_{t=1}^{T} \gamma_t(i)} \tag{32}$$

For Gaussian emissions, use the state-probability-weighted mean and variance.

Proof. The joint likelihood factorizes as:

$$P(x_{1:T}, s_{1:T}|\theta) = \pi_{s_1} A_{s_1}(x_1) \prod_{t=2}^{T} \Phi_{s_{t-1},s_t} A_{s_t}(x_t)$$

For the initial distribution:

$$\hat{\pi}_i = \arg\max_{\pi} \langle \log \pi_{s_1} \rangle_q = \arg\max_{\pi} \sum_i \log(\pi_i) P(s_1 = i|x_{1:T}) = \arg\max_{\pi} \log(\pi_i)\gamma_1(i)$$

Subject to $\sum_i \pi_i = 1$, the solution is $\hat{\pi}_i = \gamma_1(i)$.

For transitions:

$$\hat{\Phi}_{ij} = \arg\max_{\Phi} \left\langle \sum_{t=1}^{T-1} \log \Phi_{s_t,s_{t+1}} \right\rangle_q = \arg\max_{\Phi} \sum_{t=1}^{T-1} \sum_{i,j} \log(\Phi_{ij}) P(s_t = i, s_{t+1} = j|x_{1:T})$$

Subject to $\sum_j \Phi_{ij} = 1$ for each i, using Lagrange multipliers:

$$\mathcal{L} = \sum_{t,i,j} \log(\Phi_{ij}) \xi_t(i \to j) + \sum_i \lambda_i \left(1 - \sum_j \Phi_{ij}\right)$$

Taking derivative with respect to Φ_{ij}:

$$\frac{\partial \mathcal{L}}{\partial \Phi_{ij}} = \frac{\sum_t \xi_t(i \to j)}{\Phi_{ij}} - \lambda_i = 0$$

This gives $\Phi_{ij} = \frac{\sum_t \xi_t(i \to j)}{\lambda_i}$. Using the constraint $\sum_j \Phi_{ij} = 1$:

$$\lambda_i = \sum_j \sum_t \xi_t(i \to j) = \sum_t \sum_j P(s_t = i, s_{t+1} = j | x_{1:T}) = \sum_t P(s_t = i | x_{1:T}) = \sum_t \gamma_t(i)$$

Therefore:

$$\hat{\Phi}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i \to j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

For emissions (discrete case):

$$\hat{A}_{ik} = \arg\max_A \left\langle \sum_t \log A_{s_t}(x_t) \right\rangle_q = \arg\max_A \sum_{t,i} \log A_i(x_t) P(s_t = i | x_{1:T})$$

For discrete observations, $A_i(x_t) = A_{i,x_t}$, so:

$$= \arg\max_A \sum_{t,i,k} \mathbb{1}_{x_t=k} \log(A_{ik}) \gamma_t(i)$$

Subject to $\sum_k A_{ik} = 1$ for each i, the solution is:

$$\hat{A}_{ik} = \frac{\sum_{t:x_t=k} \gamma_t(i)}{\sum_t \gamma_t(i)}$$

□

Remark 3.13 (Baum-Welch Interpretation)**.** This is the Baum-Welch algorithm, which predates the more general EM algorithm. The parameter updates (Theorem 3.12) are given by ratios of expected counts: expected number of times in state i, expected number of transitions from i to j, expected number of times observation k was emitted in state i.

4 HMM Practicalities

4.1 Numerical Scaling

Remark 4.1 (Underflow Problem)**.** The conventional forward variable definition (Definition 1.9):

$$\alpha_t(i) = P(x_{1:t}, s_t = i)$$

represents a large joint probability that tends to 0 as t grows, easily causing numerical underflow.

Proposition 4.2 (Scaled Forward Variables)**.** *To address the underflow problem (Remark 4.1), define the rescaled forward variables:*

$$\begin{aligned} \bar{\alpha}_t(i) &= A_i(x_t) \sum_{j=1}^{K} \tilde{\alpha}_{t-1}(j)\Phi_{ji} \\ \rho_t &= \sum_{i=1}^{K} \bar{\alpha}_t(i) \\ \tilde{\alpha}_t(i) &= \bar{\alpha}_t(i)/\rho_t \end{aligned} \tag{33}$$

Theorem 4.3 (Scaling Factor Interpretation)**.** *The scaling factors in Proposition 4.2 satisfy:*

$$\begin{aligned} \rho_t &= P(x_t|x_{1:t-1},\theta) \\ \prod_{t=1}^{T} \rho_t &= P(x_{1:T}|\theta) \end{aligned} \tag{34}$$

and $\tilde{\alpha}_t(i) = P(s_t = i|x_{1:t},\theta)$ is the filtered posterior (cf. Corollary 1.11).

Proof. By the law of total probability:

$$P(x_t|x_{1:t-1}) = \sum_i P(x_t, s_t = i|x_{1:t-1}) = \sum_i A_i(x_t) \sum_j P(s_t = i|s_{t-1} = j)P(s_{t-1} = j|x_{1:t-1})$$

This matches the definition of ρ_t when $\tilde{\alpha}_{t-1}(j) = P(s_{t-1} = j|x_{1:t-1})$.
By induction, if $\tilde{\alpha}_{t-1}(j) = P(s_{t-1} = j|x_{1:t-1})$, then:

$$\tilde{\alpha}_t(i) = \frac{\bar{\alpha}_t(i)}{\rho_t} = \frac{P(x_t, s_t = i|x_{1:t-1})}{P(x_t|x_{1:t-1})} = P(s_t = i|x_{1:t})$$

For the likelihood:

$$P(x_{1:T}) = \prod_{t=1}^{T} P(x_t|x_{1:t-1}) = \prod_{t=1}^{T} \rho_t$$

□

4.2 Multiple Observed Sequences

Proposition 4.4 (EM with Multiple Sequences)**.** *When training on multiple sequences $\{x^{(l)}_{1:T^{(l)}}\}_{l=1}^{L}$ using the Baum-Welch algorithm (Theorem 3.12), average numerators and denominators in the parameter update ratios:*

$$\begin{aligned} \pi_i &= \frac{1}{L}\sum_{l=1}^{L} \gamma_1^{(l)}(i) \\ \Phi_{ij} &= \frac{\sum_{l=1}^{L}\sum_{t=1}^{T^{(l)}-1} \xi_t^{(l)}(ij)}{\sum_{l=1}^{L}\sum_{t=1}^{T^{(l)}-1} \gamma_t^{(l)}(i)} \\ A_{ik} &= \frac{\sum_{l=1}^{L}\sum_{t=1}^{T^{(l)}} \delta(x_t^{(l)} = k)\gamma_t^{(l)}(i)}{\sum_{l=1}^{L}\sum_{t=1}^{T^{(l)}} \gamma_t^{(l)}(i)} \end{aligned} \tag{35}$$

Algorithm 1 HMM Forward-Backward (E-step)

for $t = 1 : T$, $i = 1 : K$ **do**
  $p_t(i) \leftarrow A_i(x_t)$
end for
$\alpha_1 \leftarrow \pi \circ p_1$ ▷ $\circ$ is element-wise product
$\rho_1 \leftarrow \sum_{i=1}^{K} \alpha_1(i)$
$\alpha_1 \leftarrow \alpha_1 / \rho_1$
for $t = 2 : T$ **do**
  $\alpha_t \leftarrow (\Phi^T \alpha_{t-1}) \circ p_t$
  $\rho_t \leftarrow \sum_{i=1}^{K} \alpha_t(i)$
  $\alpha_t \leftarrow \alpha_t / \rho_t$
end for
$\beta_T \leftarrow \mathbf{1}$
for $t = T - 1 : 1$ **do**
  $\beta_t \leftarrow \Phi(\beta_{t+1} \circ p_{t+1}) / \rho_{t+1}$
end for
$\log P(x_{1:T}) \leftarrow \sum_{t=1}^{T} \log(\rho_t)$
for $t = 1 : T$ **do**
  $\gamma_t \leftarrow \alpha_t \circ \beta_t$
end for
for $t = 1 : T - 1$ **do**
  $\xi_t \leftarrow \Phi \circ (\alpha_t (\beta_{t+1} \circ p_{t+1})^T) / \rho_{t+1}$
end for

Algorithm 2 HMM Parameter Re-estimation (M-step)

for each sequence $l = 1 : L$ **do**
  Run forward-backward (Algorithm 1) to get $\gamma^{(l)}$ and $\xi^{(l)}$
end for
$\pi_i \leftarrow \frac{1}{L} \sum_{l=1}^{L} \gamma_1^{(l)}(i)$
$\Phi_{ij} \leftarrow \frac{\sum_{l=1}^{L} \sum_{t=1}^{T^{(l)}-1} \xi_t^{(l)}(ij)}{\sum_{l=1}^{L} \sum_{t=1}^{T^{(l)}-1} \gamma_t^{(l)}(i)}$
$A_{ik} \leftarrow \frac{\sum_{l=1}^{L} \sum_{t=1}^{T^{(l)}} \delta(x_t^{(l)}=k) \gamma_t^{(l)}(i)}{\sum_{l=1}^{L} \sum_{t=1}^{T^{(l)}} \gamma_t^{(l)}(i)}$

4.3 Pseudocode

5 Application: Speech Recognition

5.1 Phoneme Models

Definition 5.1 (Phoneme)**.** A phoneme is defined as the smallest unit of sound in a language that distinguishes between distinct meanings. English uses approximately 50 phonemes.

Example 5.2 (English Digit Phonemes)**.**

Zero	Z IH R OW	Six	S IH K S
One	W AH N	Seven	S EH V AX N
Two	T UW	Eight	EY T
Three	TH R IY	Nine	N AY N
Four	F OW R	Oh	OW
Five	F AY V		

Definition 5.3 (Subphonemes (Triphons))**.** Phonemes (Definition 5.1) can be further broken down into subphonemes. The standard in speech processing is to represent a phoneme by three subphonemes.

5.2 Feature Extraction

Proposition 5.4 (Speech Signal Preprocessing)**.**

1. *A speech signal is measured as amplitude over time*
2. *The signal is transformed into various features including:*
   - *Windowed Fourier or cosine transforms*
   - *Cepstral features*
3. *Each transform is a scalar function of time*
4. *All function values at time t are collected into a feature vector x_t*

5.3 HMM Speech Recognition System

Definition 5.5 (Speech Recognition Hierarchy)**.** Speech recognition involves multiple hierarchical layers:

$$\text{Speech signal} \to \text{Features} \to \text{Subphonemes} \to \text{Phonemes} \to \text{Words}$$

Algorithm 5.6 (HMM Speech Recognition)**. Training:** The HMM parameters (emission parameters and transition probabilities) are estimated from data using the Baum-Welch algorithm (Theorem 3.12), often combining both supervised and unsupervised techniques.

Recognition: Given a language signal (observation sequence $x_{1:N}$), estimate the corresponding sequence of subphonemes (states $z_{1:N}$). This is a decoding problem solved by the Viterbi algorithm (Theorem 2.10).

5.4 Speaker Adaptation

Definition 5.7 (Factory Model)**.** Training requires large amounts of data. Software is typically shipped with a model trained on a large corpus (factory settings).

Definition 5.8 (Adaptation Problem)**.**

- The factory model (Definition 5.7) represents an average speaker
- Recognition rates can be improved drastically by adapting to the specific speaker
- Before using the software, the user reads provided sentences, giving labeled training data

Proposition 5.9 (Speaker Adaptation Strategy)**.**

- *Transition probabilities are properties of the language*
- *Differences between speakers (pronunciation) are reflected in emission parameters* θ_k
- *Emission probabilities are typically multi-dimensional Gaussians*
- *Adaptation adjusts means and covariances*

Remark 5.10 (MLLR Adaptation)**.** The most widely used speaker adaptation method is Maximum Likelihood Linear Regression (MLLR), which uses regression to make small changes to the covariance matrices of the emission distributions.

6 Markov Chain Theory

6.1 Graphical Representation

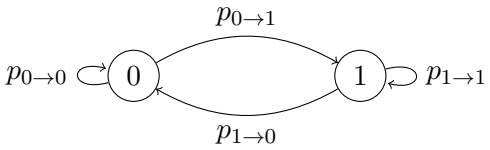
Definition 6.1 (Markov Chain Graph)**.** A (finite-state, first-order) Markov chain $X_1, X_2, \ldots$ with state space $\mathcal{X} = \{1, \ldots, d\}$ can be represented as a directed graph where:

- Vertices represent states
- Edges represent possible transitions
- Each edge $s \to t$ is weighted by the transition probability

$$p_{s \to t} := P(X_n = t | X_{n-1} = s) \tag{36}$$

Definition 6.2 (Stationarity)**.** The Markov chain (Definition 6.1) is **stationary** if $p_{s \to t}$ does not depend on the time index n.

Example 6.3 (Binary Markov Chain)**.** Consider $\mathcal{X} = \{0, 1\}$. A simple random walk might have:



Example 6.4 (Independent Coin Flips)**.** If $X_n \sim \text{Bernoulli}(p)$ independently, the transition probabilities are:

$$p_{i \to j} = \begin{cases} p & \text{if } j = 1 \\ 1 - p & \text{if } j = 0 \end{cases}$$

independent of i. This gives a chain where all states transition to the same distribution.

Example 6.5 (Breaking Independence)**.** Modifying only $p_{1\to0}$ and $p_{1\to1}$ to 0 and 1 respectively (in Example 6.4) creates dependence while maintaining the Markov property.

6.2 Transition Matrix and State Probabilities

Definition 6.6 (Transition Matrix)**.** The $d \times d$ matrix

$$\mathbf{p} = (p_{i\to j})_{i,j\le d} = \begin{pmatrix} p_{1\to 1} & \cdots & p_{d\to 1} \\ \vdots & & \vdots \\ p_{1\to d} & \cdots & p_{d\to d} \end{pmatrix} \tag{37}$$

is called the **transition matrix**. Each column is a probability distribution.

Definition 6.7 (Initial Distribution)**.** The distribution of the first state,

$$P_{\text{init}} = (P(X_0 = 1), \ldots, P(X_0 = d)) \tag{38}$$

is called the **initial distribution**.

Theorem 6.8 (Complete Specification)**.** *A stationary Markov chain (Definition 6.2) with finite state space is completely determined by the pair* $(\mathbf{p}, P_{init})$*, where* $\mathbf{p}$ *is the transition matrix (Definition 6.6) and* P_{init} *is the initial distribution (Definition 6.7).*

Theorem 6.9 (State Distribution Evolution)**.** *The marginal distribution of the chain's state after* n *steps is:*

$$P_n = \mathbf{p}^n P_{init} \tag{39}$$

Proof. For $n = 1$:

$$P_1(s_1) = P(X_1 = s_1) = \sum_{s_0\in\mathcal{X}} P(X_1 = s_1|X_0 = s_0)P_{\text{init}}(s_0) = \sum_{s_0} p_{s_0\to s_1} P_{\text{init}}(s_0) = [\mathbf{p}\cdot P_{\text{init}}](s_1)$$

By induction: $P_n = \mathbf{p}\cdot P_{n-1} = \mathbf{p}\cdot\mathbf{p}^{n-1}P_{\text{init}} = \mathbf{p}^n P_{\text{init}}$. □

6.3 Equilibrium Distributions

Definition 6.10 (Limiting Distribution)**.** If the limit exists, define:

$$P_\infty := \lim_{n\to\infty} P_n = \lim_{n\to\infty} \mathbf{p}^n P_{\text{init}} \tag{40}$$

Proposition 6.11 (Invariance of Limit)**.** *If* P_∞ *(Definition 6.10) exists, then:*

$$\mathbf{p}\cdot P_\infty = \mathbf{p}\cdot \lim_{n\to\infty} \mathbf{p}^n P_{init} = \lim_{n\to\infty} \mathbf{p}^n P_{init} = P_\infty$$

Definition 6.12 (Equilibrium Distribution)**.** A distribution P on $\mathcal{X}$ that is invariant under the transition matrix $\mathbf{p}$ (Definition 6.6) in the sense that

$$\mathbf{p}\cdot P = P \tag{41}$$

is called an **equilibrium distribution** or **invariant distribution** of the chain.

Definition 6.13 (Aperiodic Chain)**.** A stationary Markov chain is **aperiodic** if

$$P(X_n = s|X_{n-1} = s) = p_{s\to s} > 0 \quad \text{for all } s \in \mathcal{X}$$

That is, the transition matrix has non-zero diagonal entries.

Definition 6.14 (Irreducible Chain)**.** A chain is **irreducible** if there is a path with non-zero probability from each state to every other state in the transition graph (Definition 6.1).

Theorem 6.15 (Existence and Uniqueness of Equilibrium)**.** *If a first-order, stationary Markov chain with finite state space is aperiodic (Definition 6.13) and irreducible (Definition 6.14), then:*

1. *The limit distribution P_∞ (Definition 6.10) exists*
2. *The limit distribution is also the equilibrium distribution (Definition 6.12)*
3. *The equilibrium distribution is unique*
4. *The equilibrium does not depend on the initial distribution*

Remark 6.16 (Why Irreducibility?)**.** Without irreducibility, multiple equilibria can exist. For example, in a chain with disconnected components, each component can have its own equilibrium, and which emerges depends on where the chain starts.

Remark 6.17 (Why Aperiodicity?)**.** Without aperiodicity, the distribution P_n may oscillate and fail to converge. For example, a chain alternating deterministically between two states has P_n oscillating between two distributions.

6.4 Computing the Equilibrium

Algorithm 6.18 (Power Method)**.** If the transition matrix $\mathbf{p}$ makes the chain irreducible and aperiodic, we can approximate P_∞ (Theorem 6.15) by P_n:

1. Initialize with any distribution P_{init} (e.g., uniform)
2. Repeat: $P_{n+1} = \mathbf{p} \cdot P_n$
3. Terminate once $\|P_{n+1} - P_n\| < \tau$ for some small $\tau > 0$

Proposition 6.19 (Eigenstructure)**.**

- *The equation $P = \mathbf{p} \cdot P$ (Definition 6.12) means $P = P_\infty$ is an eigenvector of $\mathbf{p}$ with eigenvalue 1*
- *If $\mathbf{p}$ is irreducible and aperiodic (Theorem 6.15), 1 is the largest eigenvalue*
- *Convergence speed of the power method (Algorithm 6.18) depends on the spectral gap (ratio between largest and second-largest eigenvalue)*

7 Application: Web Search and PageRank

7.1 The Web Graph

Definition 7.1 (Web Graph)**.** The link structure of the web is represented by the adjacency matrix:

$$A = (A_{ij})_{i,j \le \#\text{pages}} \quad \text{where} \quad A_{ij} = \begin{cases} 1 & \text{page } i \text{ links to page } j \\ 0 & \text{otherwise} \end{cases}$$

Vertices represent pages, edges represent links. The graph is directed.

Definition 7.2 (Simple Random Walk)**.** Let G be a directed graph (e.g., Definition 7.1) with d vertices. Generate a sequence $X_0, X_1, \ldots$ of vertices:

1. Select a vertex X_0 in G uniformly at random
2. For $n = 1, 2, \ldots$, select X_n uniformly at random from the children of X_{n-1} in the graph

This defines a Markov chain (Definition 6.1) with state space = vertex set, and:

$$P_{\text{init}} = \left(\frac{1}{d}, \ldots, \frac{1}{d}\right), \quad p_{i \to j} = \begin{cases} \frac{1}{\#\text{ edges out of } i} & \text{if } i \text{ links to } j \\ 0 & \text{otherwise} \end{cases}$$

Algorithm 7.3 (PageRank Algorithm)**.** (Approximately) compute P_∞ (Definition 6.10) for simple random walk (Definition 7.2) on the web graph.

7.2 Interpretation of PageRank

Definition 7.4 (Ranking Problem)**.** The first step in internet search is query matching:

1. User enters a search query (string of words)
2. Search engine determines all matching web pages
3. This is a large set (typically tens or hundreds of millions)
4. Results are only useful if the "best" links can be filtered out

Proposition 7.5 (PageRank Solution)**.**

- *PageRank uses $P_\infty(v)$ (Definition 6.10) as the score of web page v*
- *Pages are ranked by decreasing score*
- *Uses: (i) popularity as proxy for quality/usefulness, (ii) incoming links as proxy for popularity*
- *P_1 would measure how often a page is linked*
- *P_2 weights this by how often the linking page is linked (by Theorem 6.9)*
- *P_∞ is the long-run equilibrium of this recursive popularity measure*

Remark 7.6 (Random Surfer Interpretation)**.** The PageRank paper interpreted $P_\infty(v)$ as the probability that a "random web surfer" would end up on page v after randomly following a large number of links. The start page does not matter as $n \to \infty$ (by Theorem 6.15).

7.3 PageRank Implementation

Definition 7.7 (Raw Web Transition Matrix)**.** Simple random walk on the web graph (Definition 7.1) has transition matrix T with:

$$T_{ij} := \begin{cases} \frac{1}{\#\text{ edges out of } i} & \text{if } i \text{ links to } j \\ 0 & \text{otherwise} \end{cases}$$

This is typically not irreducible (pages with no outgoing links create absorbing states).

Definition 7.8 (PageRank Matrix)**.** PageRank uses the regularized transition matrix:

$$p_{ij} := (1 - \alpha)T_{ij} + \frac{\alpha}{d} \tag{42}$$

for some small $\alpha \in (0, 1)$ (typically $\alpha = 0.15$), where T is as in Definition 7.7. This makes $\mathbf{p}$ both irreducible (Definition 6.14) and aperiodic (Definition 6.13), so Theorem 6.15 applies.

Algorithm 7.9 (PageRank Computation)**.**

1. Approximate equilibrium by the power method (Algorithm 6.18)
2. Since web changes, re-run every few days
3. Use previous equilibrium as initial distribution for warm start